Linux - Paparazzi - CV – Bebop

C. De Wagter – G. de Croon
MAVLab, Faculty of Aerospace Engineering
TU Delft, the Netherlands

TU Delft

3 Crash Courses to get you started

- Linux / Paparazzi-UAV
- Programming in C
- GIT

Overall goal: "Do research with MAV"

Goals this lecture

- Be able to Run (Install) Linux, Gazebo, Paparazzi
- Know where to find information to get started with paparazzi
- Understand the main structure of Paparazzi-UAV
- Understand the difficulties of Embedded Computer Vision

Practical step-by-step tutorial:
paparazzi_crash_course.pdf

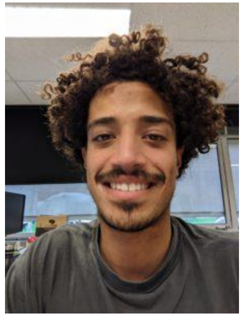
paparazzi_crash_course.pdf

https://tudelft.github.io/coursePaparazzi/

Get Help

- Practical sessions
- 4 TA's
- CyberZoo testing timeslots

Erik van der Horst
e.vanderhorst@tudelft.nl



Robin Ferede
r.ferede@tudelft.nl

Kevin Malkow
k.malkow@student.tudelft.nl

Reinier Zwikker
r.j.n.zwikker@student.tudelft.nl

Linux

Linux

- Family of open-source software operating systems
- Kernel started by Linus Torvalds
- Servers, routers, embedded systems, smartphones, supercomputers, android, ...
- We will work with **Ubuntu 22.04 LTS** (18.04, 20.04 supported and 24.04 partially supported: use 20.04 for open-cvbebop)
- Install most open-source software with:

```
sudo apt-get install XXXX
sudo apt-cache search XXXXX
```

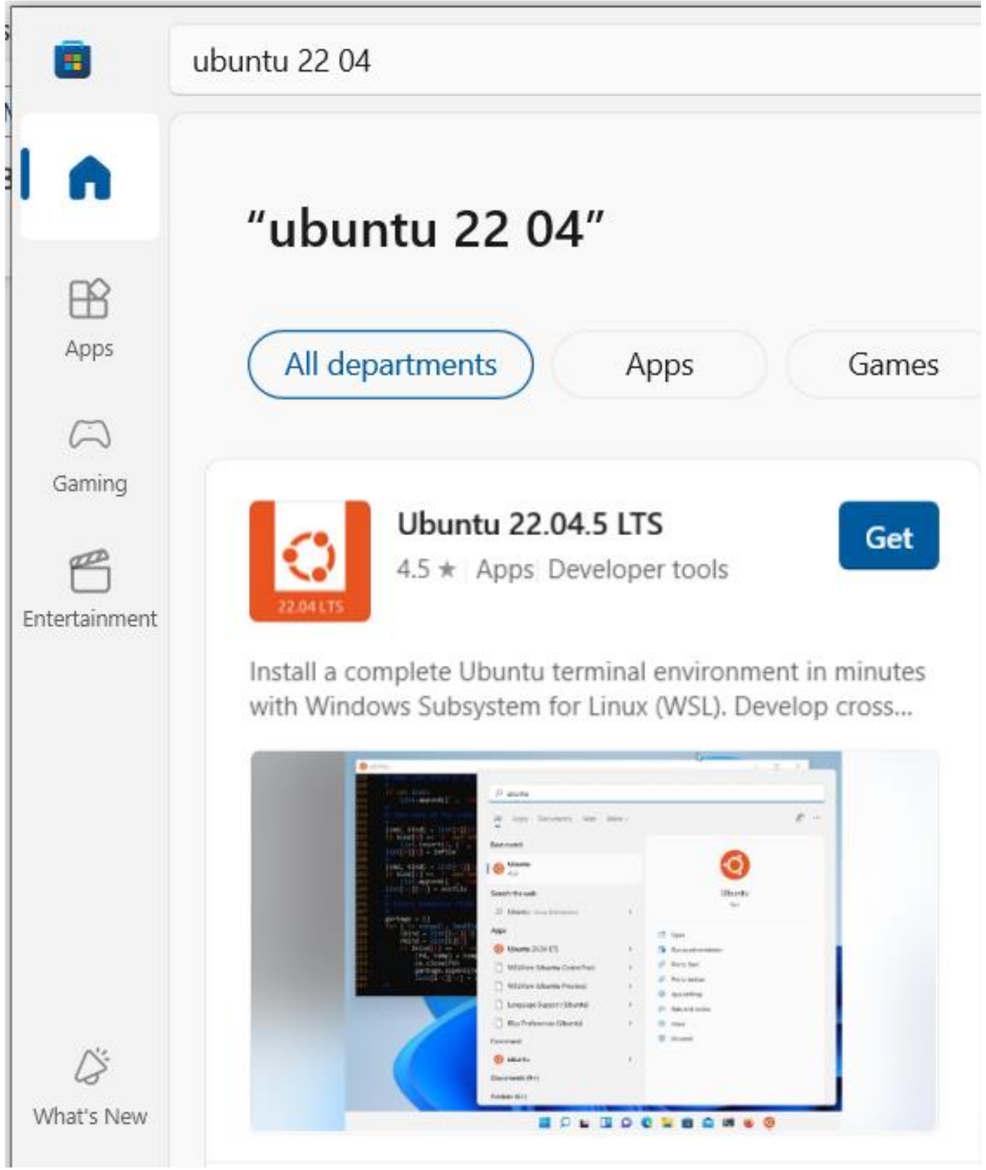
Running Linux

- 1) Native (dual boot)
  - Pro: FAST, GPU, Full-option
  - Con: Some driver problems can occur, UEFI secure-boot, you can delete your windows if you miss-click
- 2) WSL(2): Ubuntu in windows (e.g. AE4353)
- 3) Run in virtual machine
  - Con: **Slow**, some networking issues (hidden behind router)
- 4) Run from special USB 3.0 stick
  - Con: **Slow** boot/load/compile

2) WSL

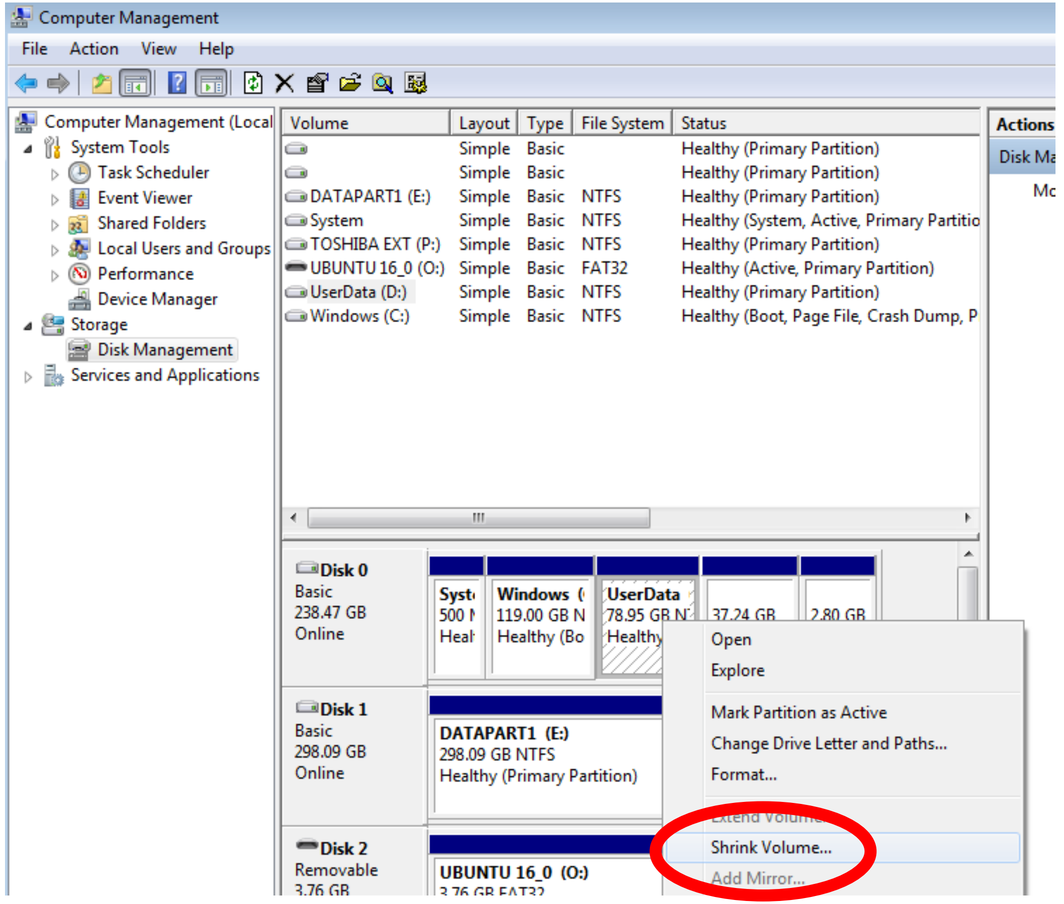
- App Store
- Click Ubuntu 22.04

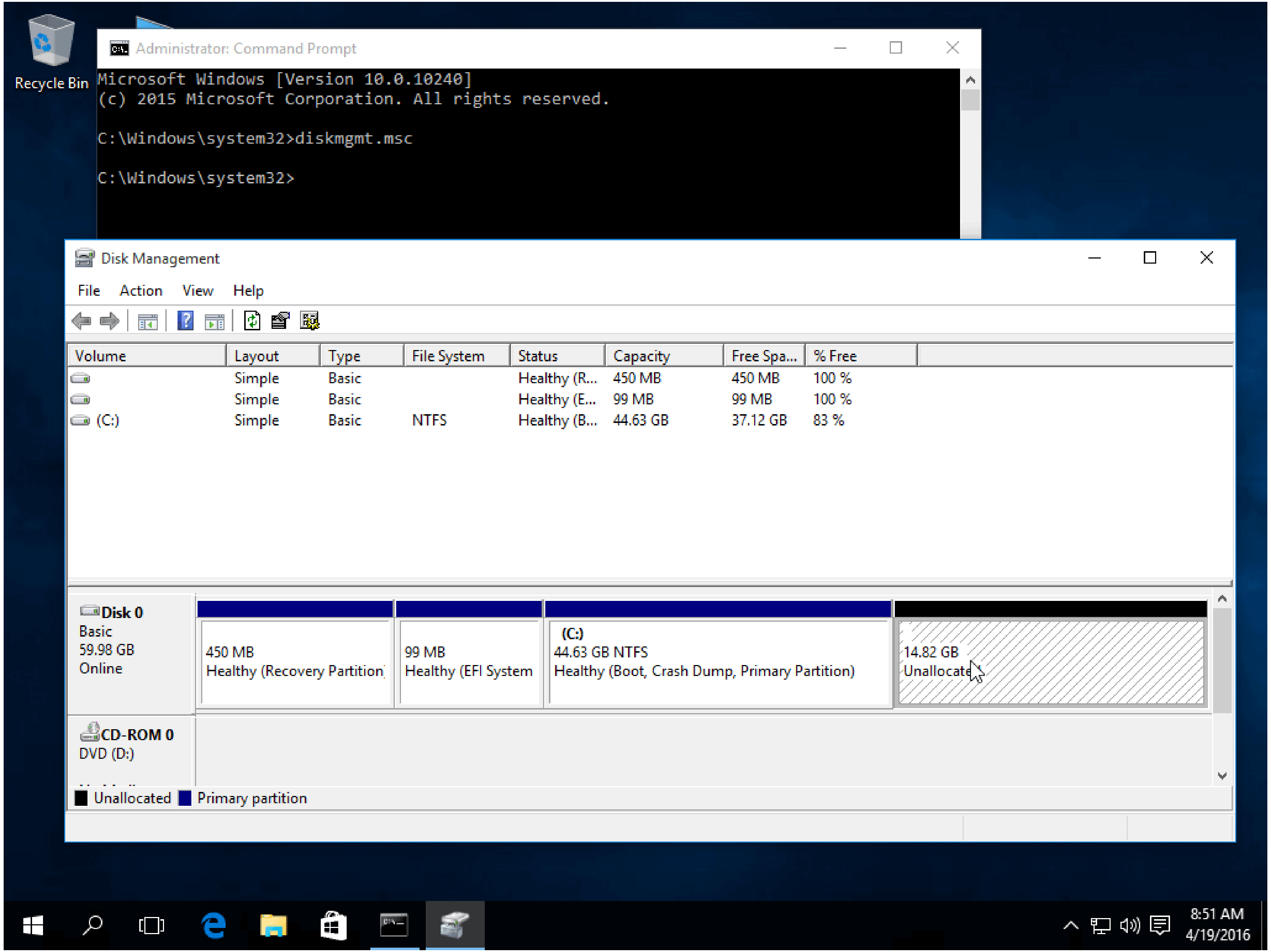
- More info:
- https://learn.microsoft.com/en-us/windows/wsl/install



1) Dual Boot

- Make some space
  - –Shrink
  - –Absolute minimum 21GB
  - –Recommended 40GB
- Win 8 & 10
  - –Disable secure-boot




Recycle Bin
Administrator: Command Prompt
Microsoft Windows [Version 10.0.10240]
(c) 2015 Microsoft Corporation. All rights reserved.
C:\Windows\system32>diskmgmt.msc
C:\Windows\system32>
Disk Management
File Action View Help
Volume Layout Type File System Status Capacity Free Spa... % Free
Simple Basic Healthy (R... 450 MB 450 MB 100 %
Simple Basic Healthy (E... 99 MB 99 MB 100 %
(C:) Simple Basic NTFS Healthy (B... 44.63 GB 37.12 GB 83 %
Disk 0
Basic
59.98 GB
Online
450 MB
Healthy (Recovery Partition)
99 MB
Healthy (EFI System
(C:)
44.63 GB NTFS
Healthy (Boot, Crash Dump, Primary Partition)
14.82 GB
Unallocate
CD-ROM 0
DVD (D:)
Unallocated
Primary partition
8:51 AM
4/19/2016

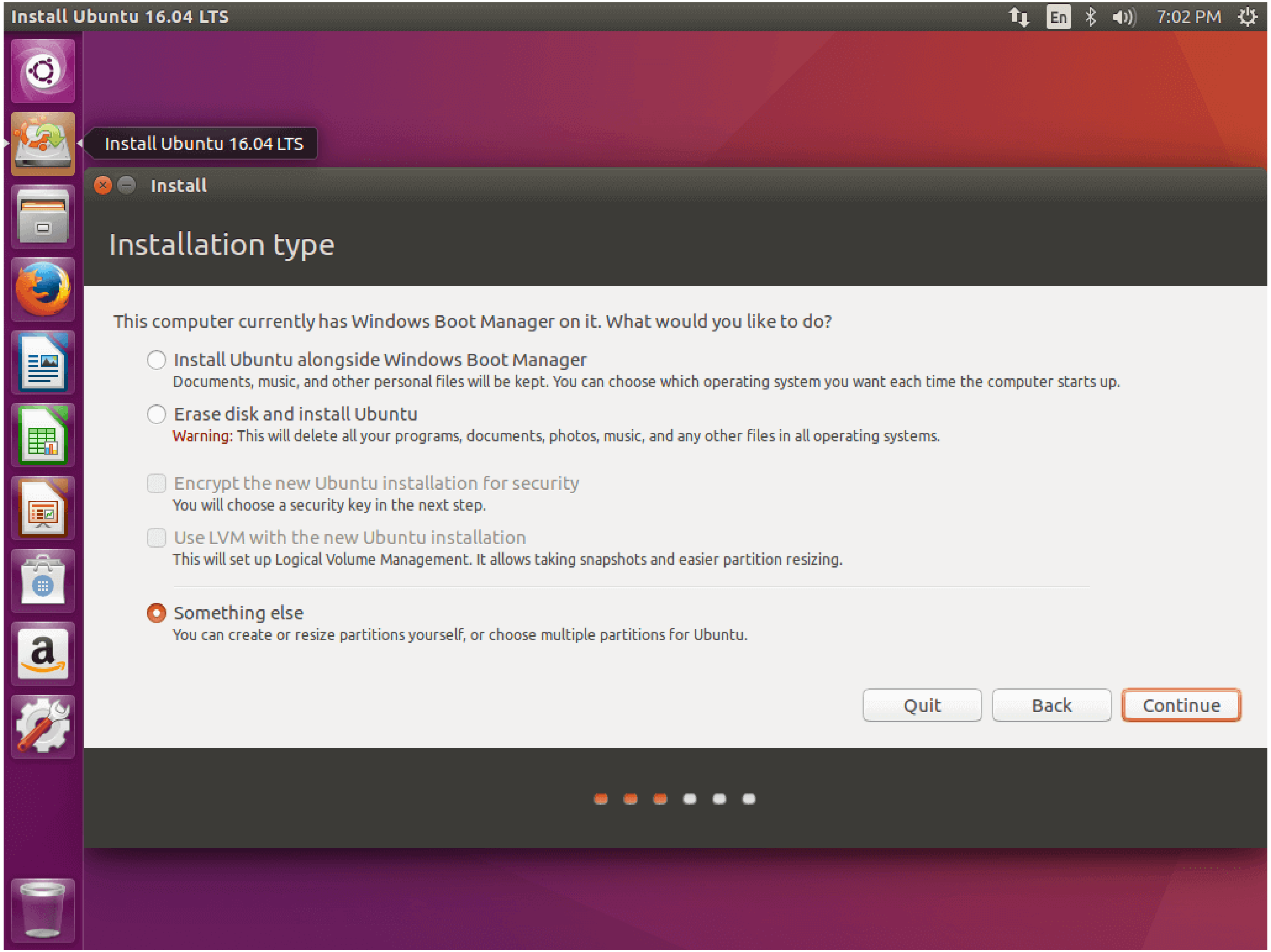
TU Delft

Installation CD/USB

- Google ubuntu and download ISO
  - Use RUFUS or similar to put iso to USB
  - Boot from USB

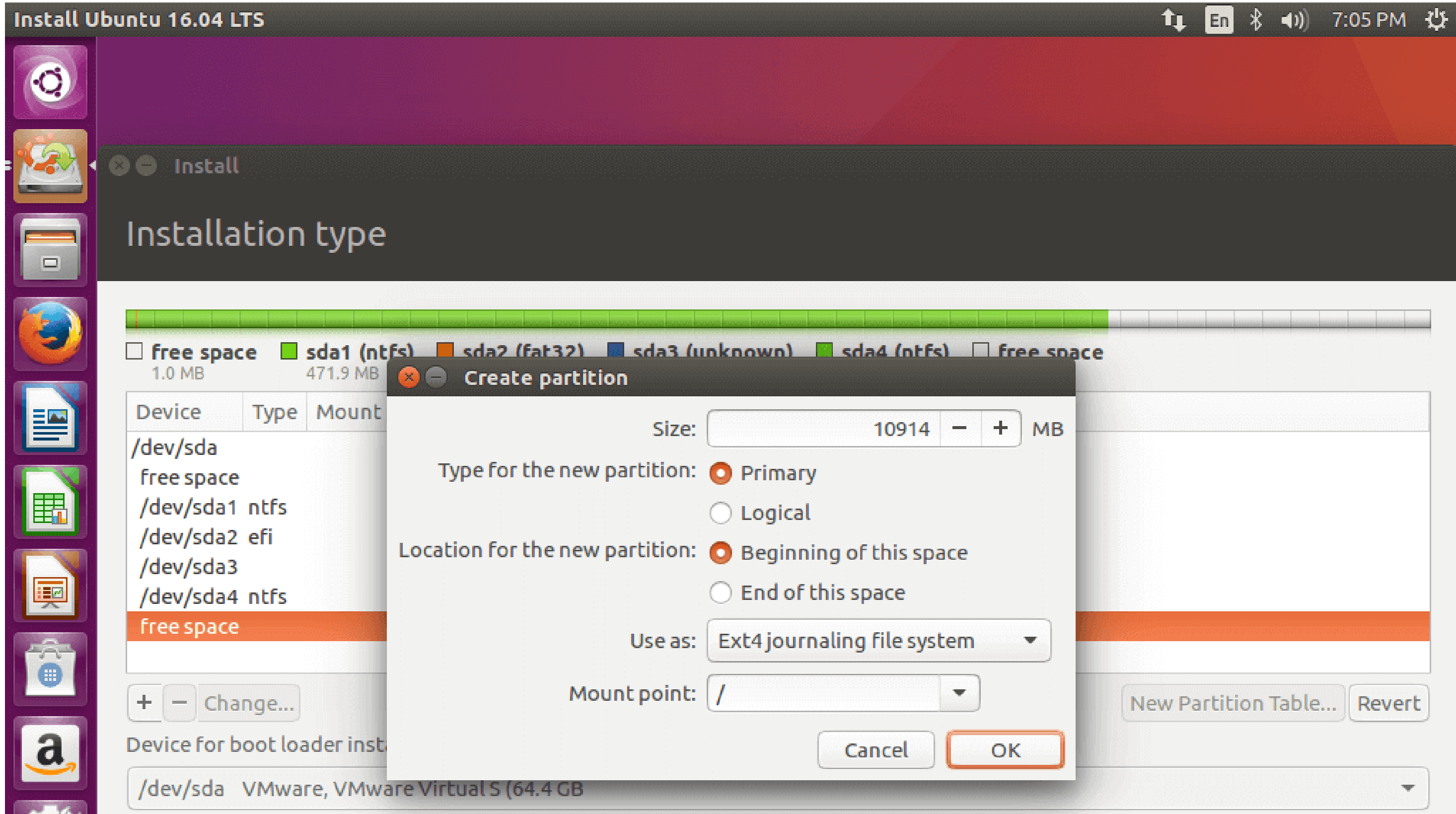
- Help with dual-boot:
  - https://itsfoss.com/install-ubuntu-1404-dual-boot-mode-windows-8-81-uefi/

TU Delft


Install Ubuntu 16.04 LTS
En
7:02 PM
Install Ubuntu 16.04 LTS
Install
Installation type
This computer currently has Windows Boot Manager on it. What would you like to do?
Install Ubuntu alongside Windows Boot Manager
Documents, music, and other personal files will be kept. You can choose which operating system you want each time the computer starts up.
Erase disk and install Ubuntu
Warning: This will delete all your programs, documents, photos, music, and any other files in all operating systems.
Encrypt the new Ubuntu installation for security
You will choose a security key in the next step.
Use LVM with the new Ubuntu installation
This will set up Logical Volume Management. It allows taking snapshots and easier partition resizing.
Something else
You can create or resize partitions yourself, or choose multiple partitions for Ubuntu.
Quit
Back
Continue

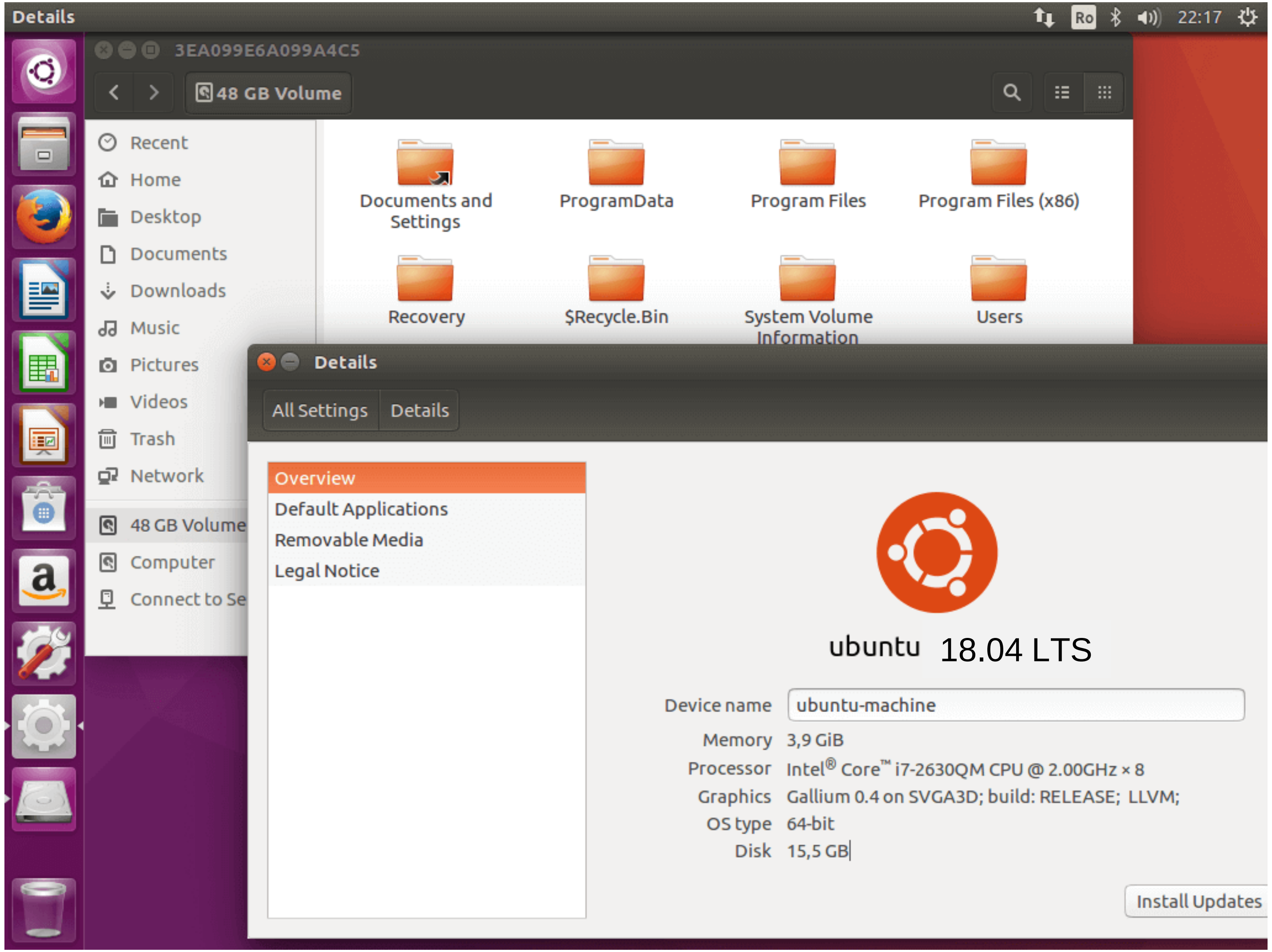
Install on free partition

- Make an EXT4 journaling file system in the free space
- Mount point = /
- Size = as much as you have
- If you are low on RAM, use some of the free space to make a SWAP drive

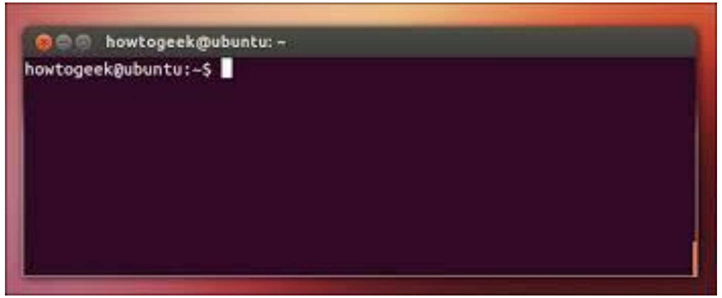


TUDelft

Welcome to Ubuntu



Linux Terminal Checklist



- Use the **tab** key (Auto-completion)
- **cd** XXXX (change directory)
- **ls** (list files), **ll** | more (list details)
- **mv** or **cp** or **mkdir** (move or copy or make directory)
- **rm -rf** XXXX (remove, recursive, forced)
- ~/ (home folder)
- **chmod** +x XXXX and chown
- **ps** (running programs)
- **killall** -9 XXXX
- git branch -r **| grep** tudelft
- **sudo** SOMETHING (do as administrator)
- sudo **apt-get install** XXXXX
- **telnet** IP, ftp IP, ssh IP, ...

TUDelft

Paparazzi

Paparazzi-UAV Project

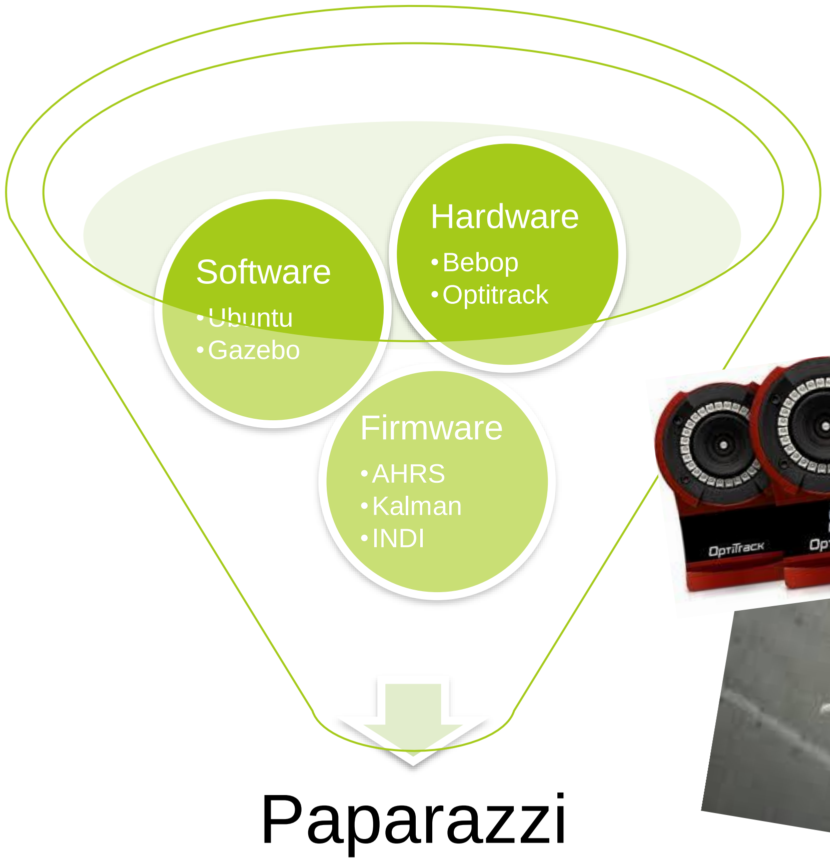
- *paparazziuav.org*

Paparazzi-UAV: WHY?

- 45++ boards (smallest – largest)
- 150++ sensors-types/actuators/filters
- 100% open-source/open-hardware
- Flight-plans are programs/scripts
- 1^{st} ARM, 1^{st} Hybrid, Smallest, CV, …
- Computer vision incorporated
  - Light Embedded CV (MAV!)
- Direct access to sensors and camera's of the bebop drone (No SDK)

TUDelft

Paparazzi-UAV Project

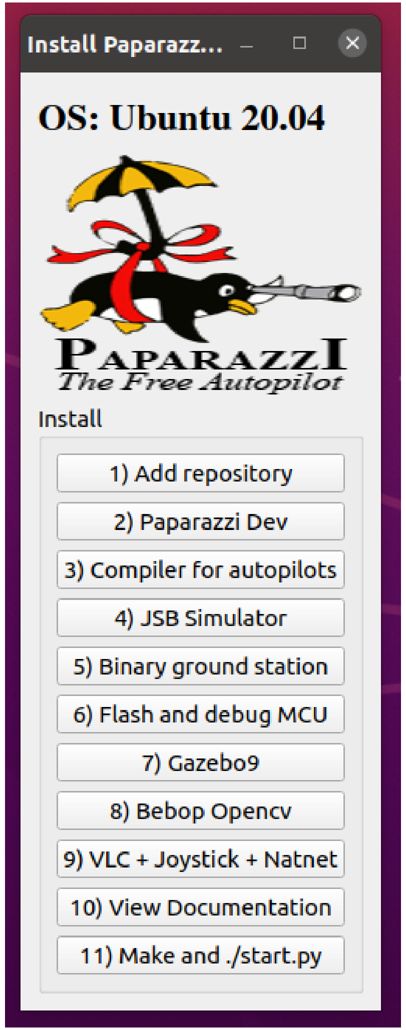


Install Paparazzi http://www.paparazziuav.org/

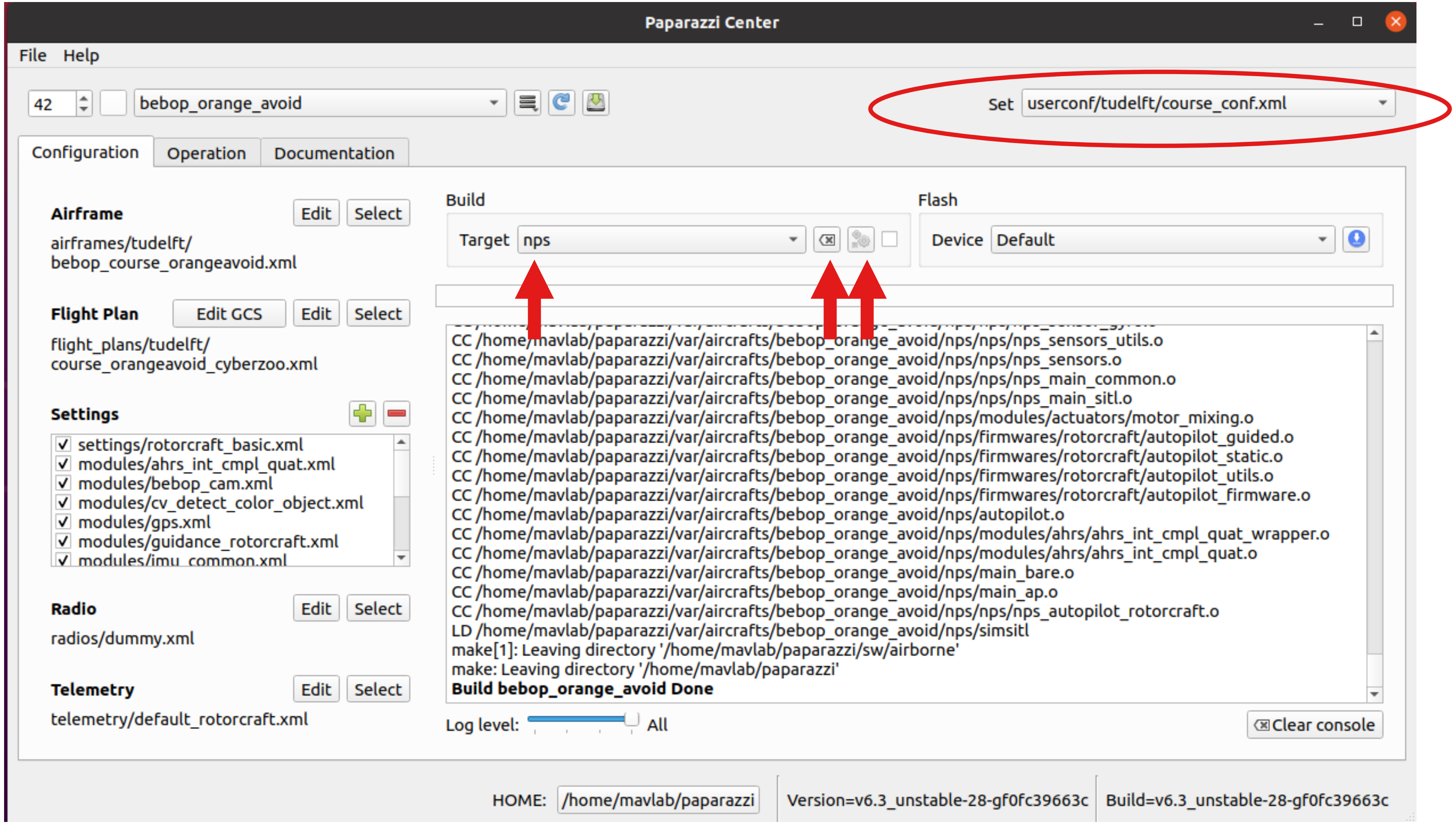
- Run Ubuntu 22.04 (or 18.04 or 20.04)

- git clone https://github.com/tudelft/paparazzi.git
- cd ./paparazzi
- ./install.sh

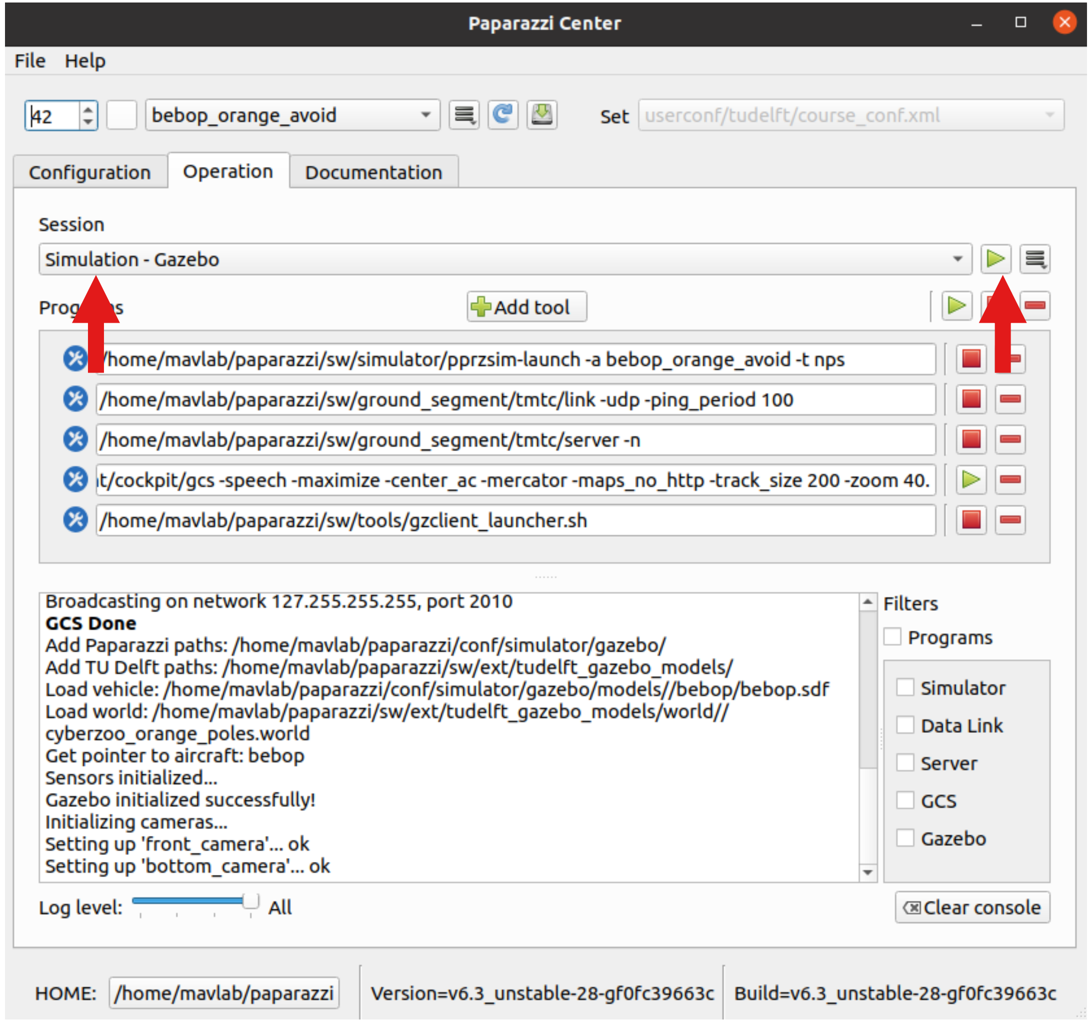
- At least: 1, 2, 3, 4, 5, 7, 9 + make&start



Build the New Paparazzi Simulator (NPS)

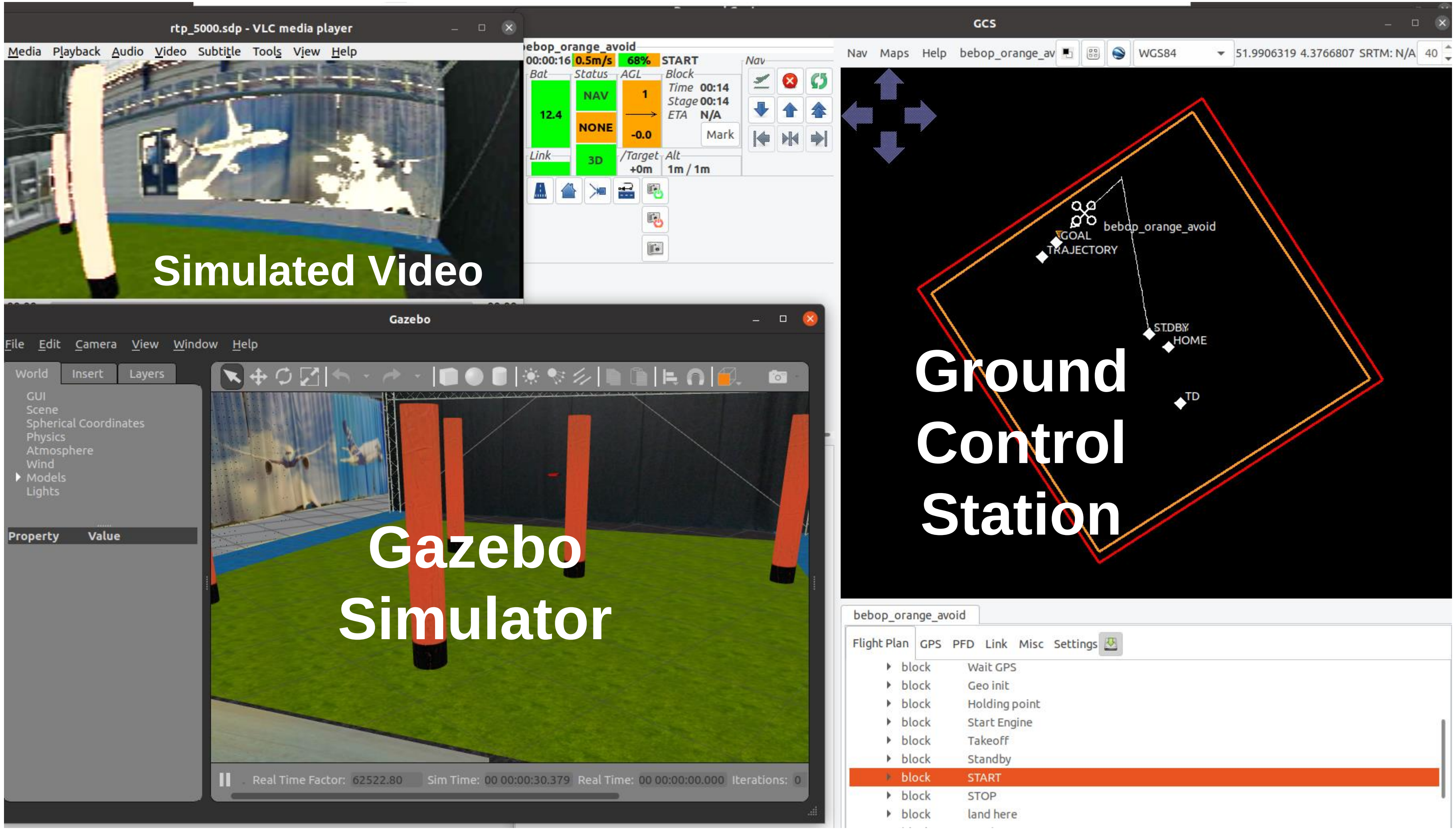


Launch Simulation



TUDelft

Goal: week 1: run simulations



Where to find help?

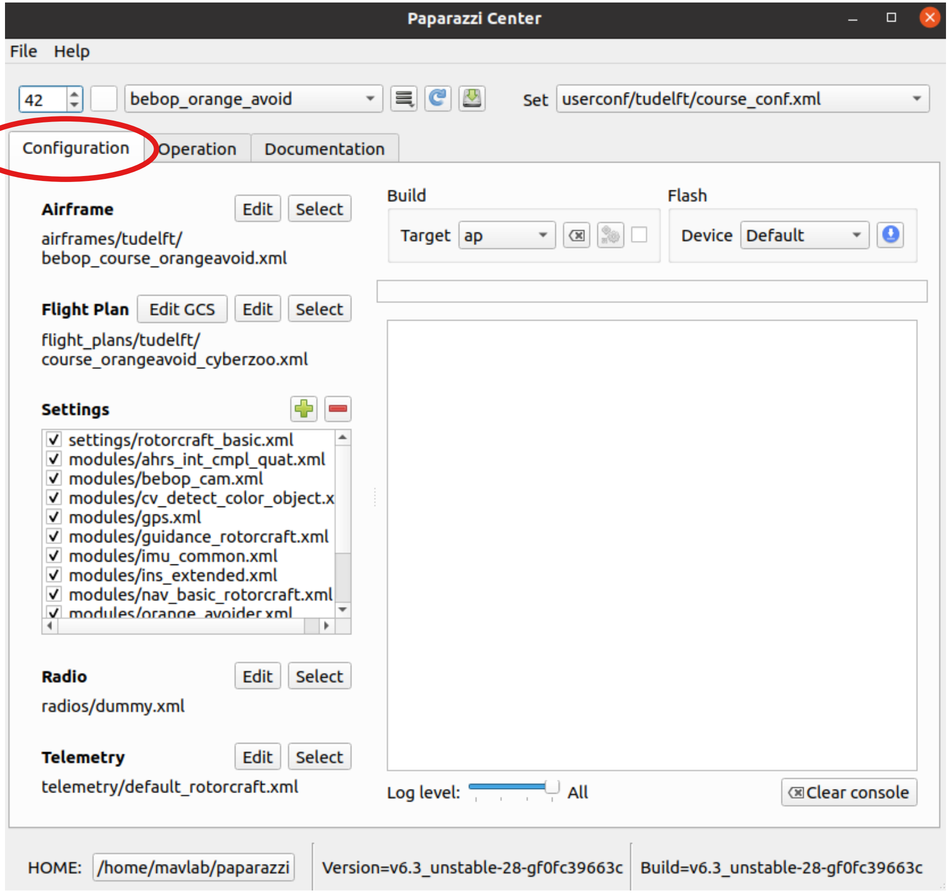
https://tudelft.github.io/coursePaparazzi/

1) Follow course pdf
2) Ask teammates
3) Google for laptop
   or hardware specific problems
4) Ask our TA's during practical sessions

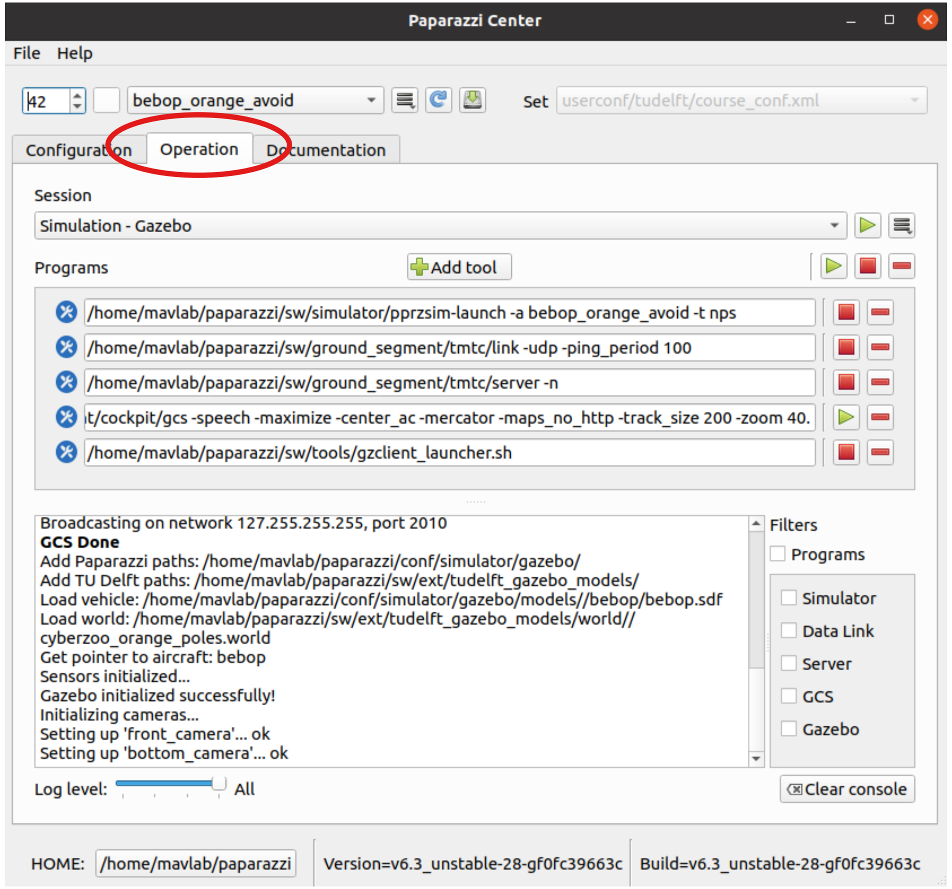
TUDelft

Paparazzi-UAV Project Structure

AIR



GROUND



TU Delft

Structure

AIR

- conf.xml
  (list of airframes)
  - airframe.xml
    - module.xml (conf/modules/)
      - *.h,*.c-files
      - gains/settings/…

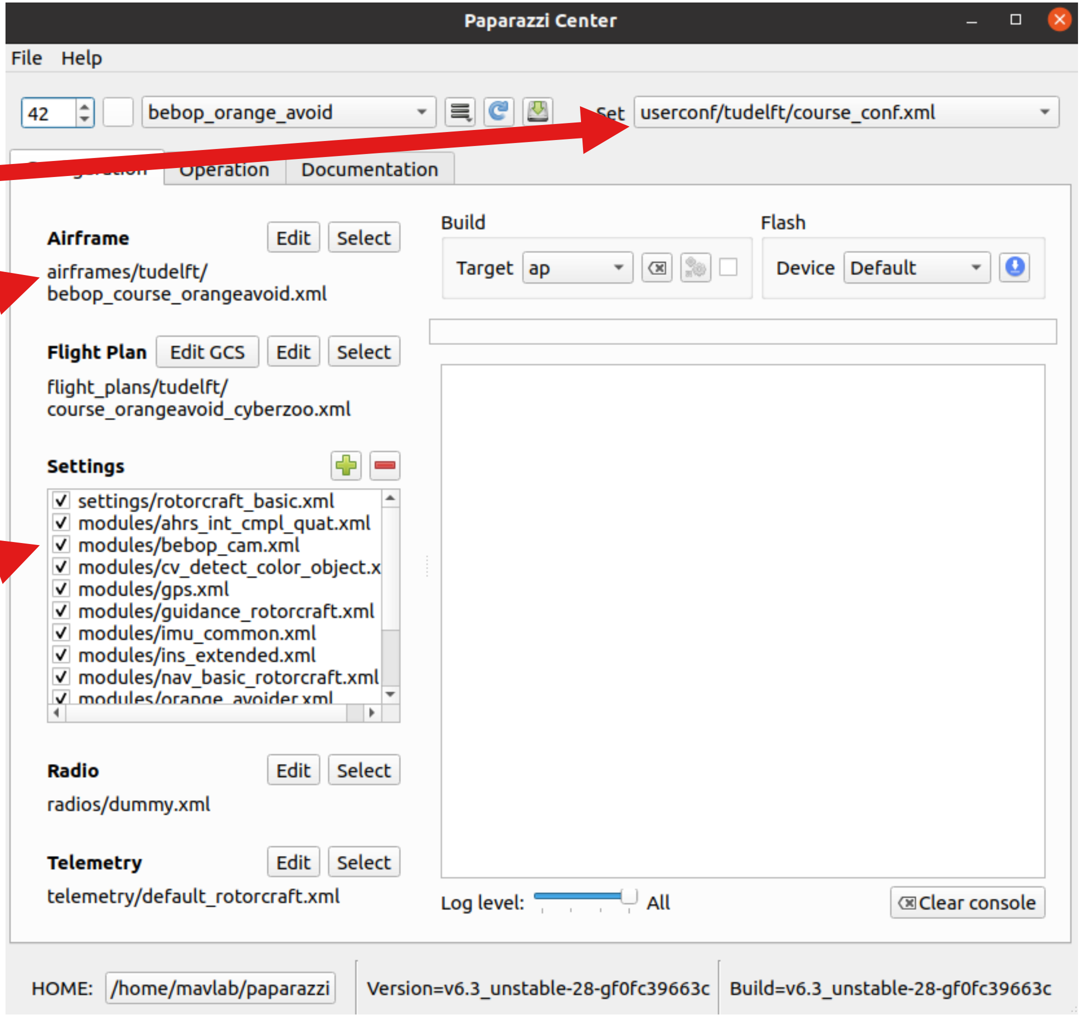
GROUND

- controlpanel.xml
  - Server (IVY)
  - Link
  - Messages
  - Plotter
  - GCS
  - natnet
  - …

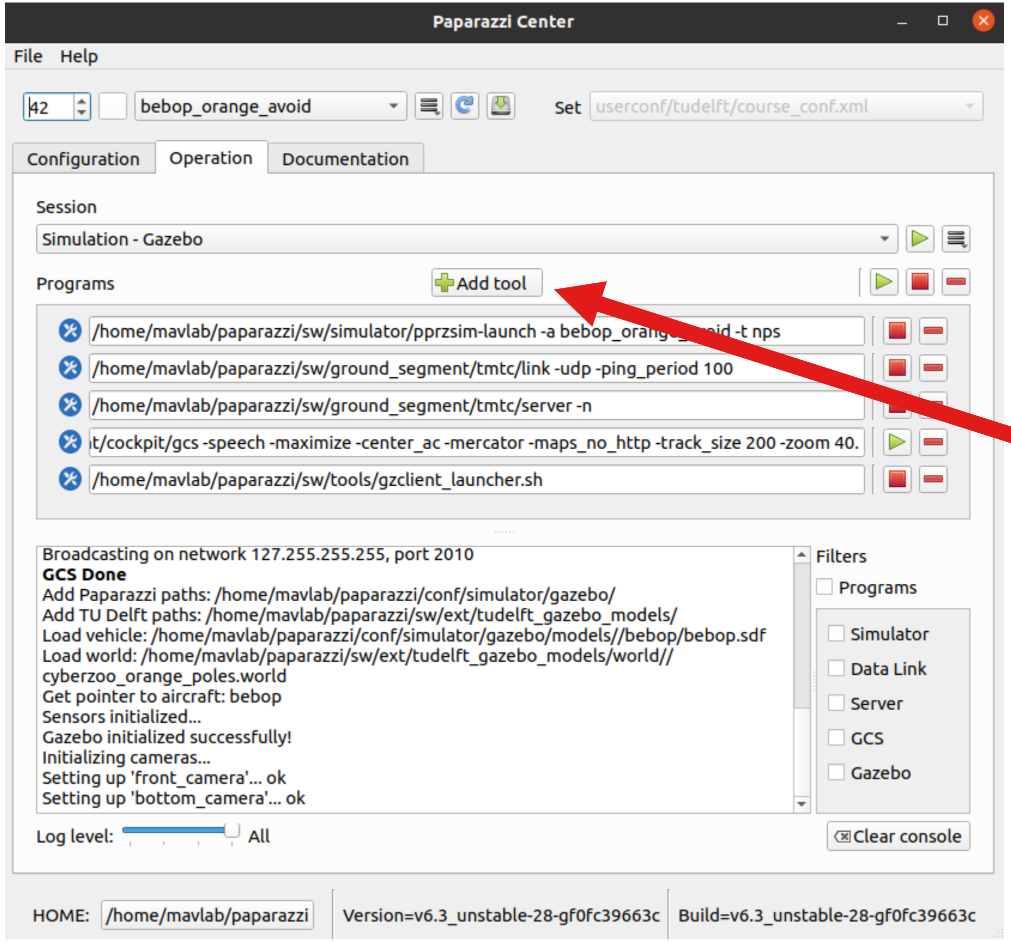
Structure

AIR

- conf.xml
  (list of airframes)
  - –airframe.xml
    - •module.xml (conf/modules/)
      - –*.h,*.c-files
      - –gains/settings/…



Structure



GROUND

- controlpanel.xml
  - Server (IVY)
  - Link
  - Messages
  - Plotter
  - GCS
  - natnet
  - …

AIR

CONF - AIRFRAME - MODULE

```
<conf>
  <aircraft
   name="ARDrone2_default"
   ac_id="10"
   airframe="airframes/exampl
   radio="radios/dummy.xml"
   telemetry="telemetry/defau
   flight_plan="flight_plans/
   settings="settings/rotorcr
   settings_modules="modules/
modules/stabilization_int_qua
   gui_color="#ffffd633d633"
  />
  <aircraft
   name="ARDrone2_flip"
   ac_id="14"
   airframe="airframes/TUDELF
   radio="radios/dummy.xml"
   telemetry="telemetry/defau
   flight_plan="flight_plans/
   settings="settings/rotorcr
   settings_modules="modules/
guidance_rotorcraft.xml modul
   gui_color="#ffffcd49cd49"
  />
```

```
<!DOCTYPE airframe SYSTEM "../air

<airframe name="ardrone2">

  <firmware name="rotorcraft">
    <target name="ap" board="ardr

    <target name="nps" board="pc"
      <module name="fdm" type="ga
    </target>

    <define name="USE_SONAR" valu

        <!-- Subsystem section --
    <module name="telemetry" type
    <module name="radio_control"
    <module name="motor_mixing"/>
    <module name="actuators" type
    <module name="imu" type="ardr
    <!-- gps: "ublox" or change t
    <module name="gps" type="ublo
    <module name="stabilization"
    <module name="ahrs" type="int
    <module name="ins" type="exte
  </firmware>
```

```
<!DOCTYPE module SYSTEM "module.

<module name="demo_module">
  <doc>
    <description>Demo module</de
  </doc>
  <header>
    <file name="demo_module.h"/>
  </header>
  <init fun="init_demo()"/>
  <periodic fun="periodic_1Hz_de
  <periodic fun="periodic_10Hz_d
  <makefile>
    <raw>
#Example of RAW makefile part
    </raw>
    <define name="DEMO_MODULE_LE
    <file name="demo_module.c"/>
  </makefile>
  <makefile target="demo">
    <define name="SOME_FLAG"/>
    <configure name="SOME_DEFINE
  </makefile>
</module>
```

Module code is in ./sw/airborne/modules/

Modules

- DOCUMENTATION
- SETTINGS
- FUNCTIONS
  - INIT
  - PERIODIC
  - EVENT
- FILES
  - *.h, *.c

```
<!DOCTYPE module SYSTEM "module.dtd">

<module name="demo_module">
  <doc>
    <description>Demo module</description>
  </doc>
  <header>
    <file name="demo_module.h"/>
  </header>
  <init fun="init_demo()"/>
  <periodic fun="periodic_1Hz_demo()" freq="1." sta
  <periodic fun="periodic_10Hz_demo()" period="0.1"
  <makefile>
    <raw>
#Example of RAW makefile part
    </raw>
    <define name="DEMO_MODULE_LED" value="2"/>
    <file name="demo_module.c"/>
  </makefile>
  <makefile target="demo">
    <define name="SOME_FLAG"/>
    <configure name="SOME_DEFINE" value="bla"/>
  </makefile>
</module>
```

Module code is in ./sw/airborne/modules/

TUDelft

PPRZLINK – Telemetry messages

- Separate project
- git submodule

  ./sw/ext/pprzlink

```xml
<message name="ATTITUDE" id="6">
  <field name="phi"   type="float" unit="rad" alt_unit="deg"/>
  <field name="psi"   type="float" unit="rad" alt_unit="deg"/>
  <field name="theta" type="float" unit="rad" alt_unit="deg"/>
</message>

<message name="IR_SENSORS" id="7">
  <field name="ir1" type="int16"/>
  <field name="ir2" type="int16"/>
  <field name="longitudinal" type="int16"/>
  <field name="lateral" type="int16"/>
  <field name="vertical" type="int16"/>
</message>
```

- Use DEBUG message to send data
- Or create your own messages:

  Copy the message definitions

  from: ./sw/ext/pprzlink/message_definitions/1.0/messages.xml

  to: ./conf/messages.xml

  and “make” paparazzi again

Paparazzi-checklist

you should understand:

What is in **conf.xml** and **controlpanel.xml** (separate tabs)

Airframe.xml:

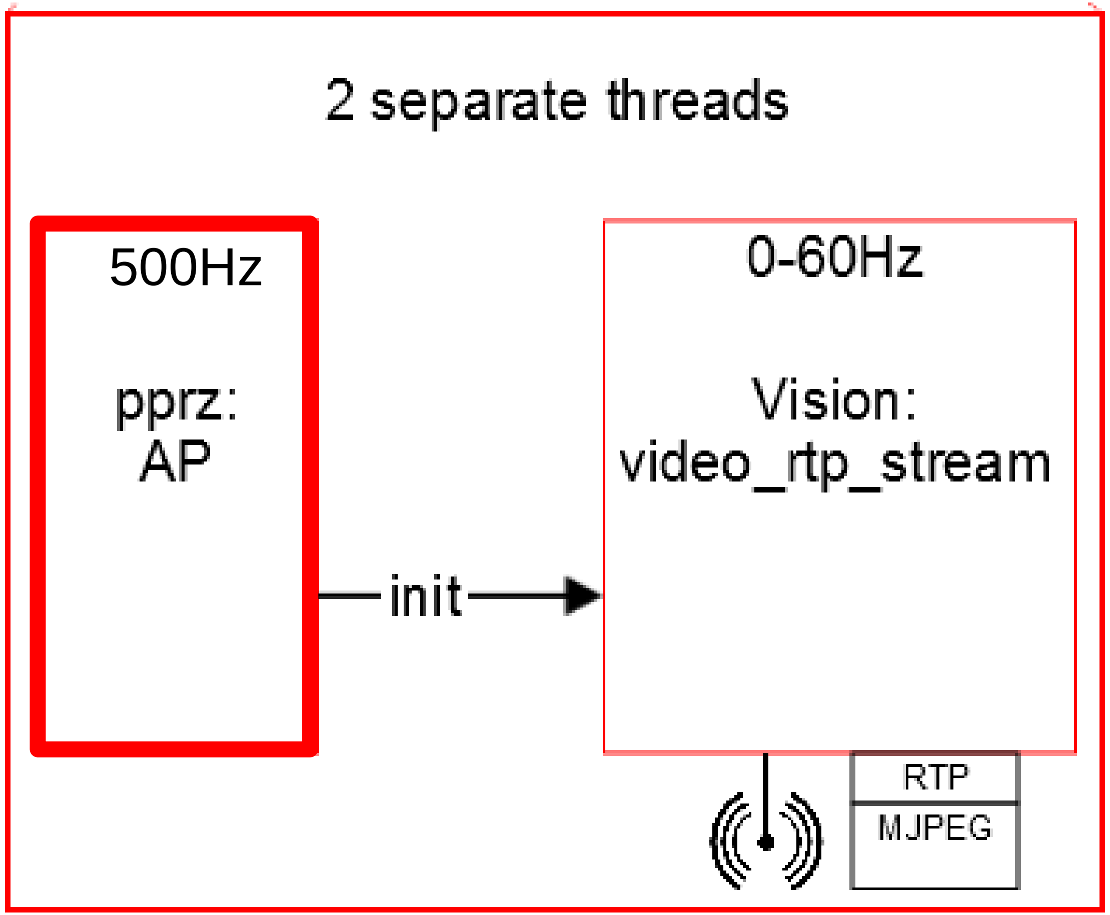
- target = either the autopilot **AP** or new pprz simulator **NPS**
- **module**s in airframe.xml -> able to find the module *.c files
- code in ./sw/airborne/modules/ ...
- *define* -> constant parameter in c-code
- Know that submodules (e.g. pprzlink) are in: ./sw/ext/...

TUDelft

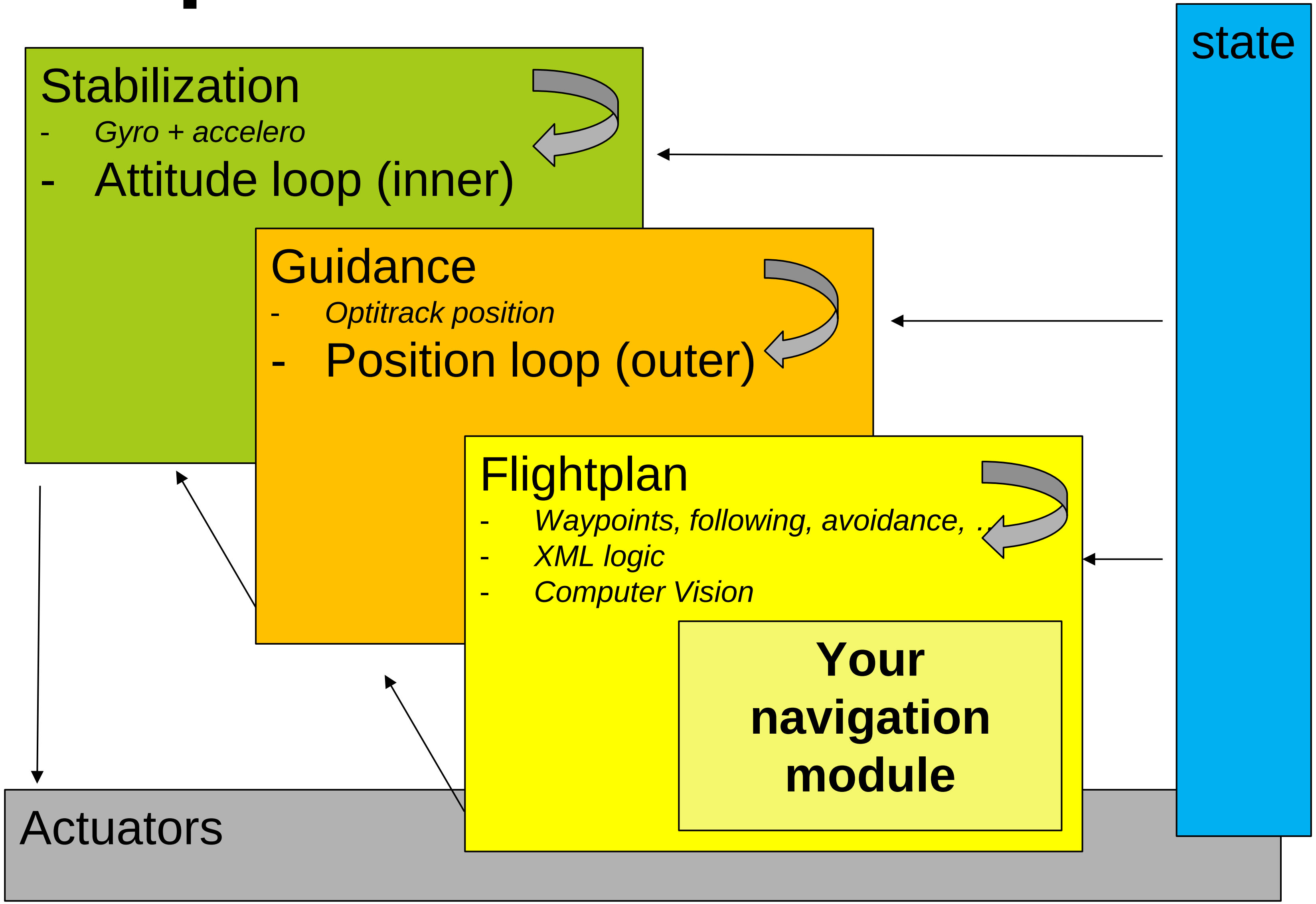
Onboard Code Structure

Flight Code: multi-threading

- Autopilot
  - 500Hz
- Vision
  - 30Hz (max)

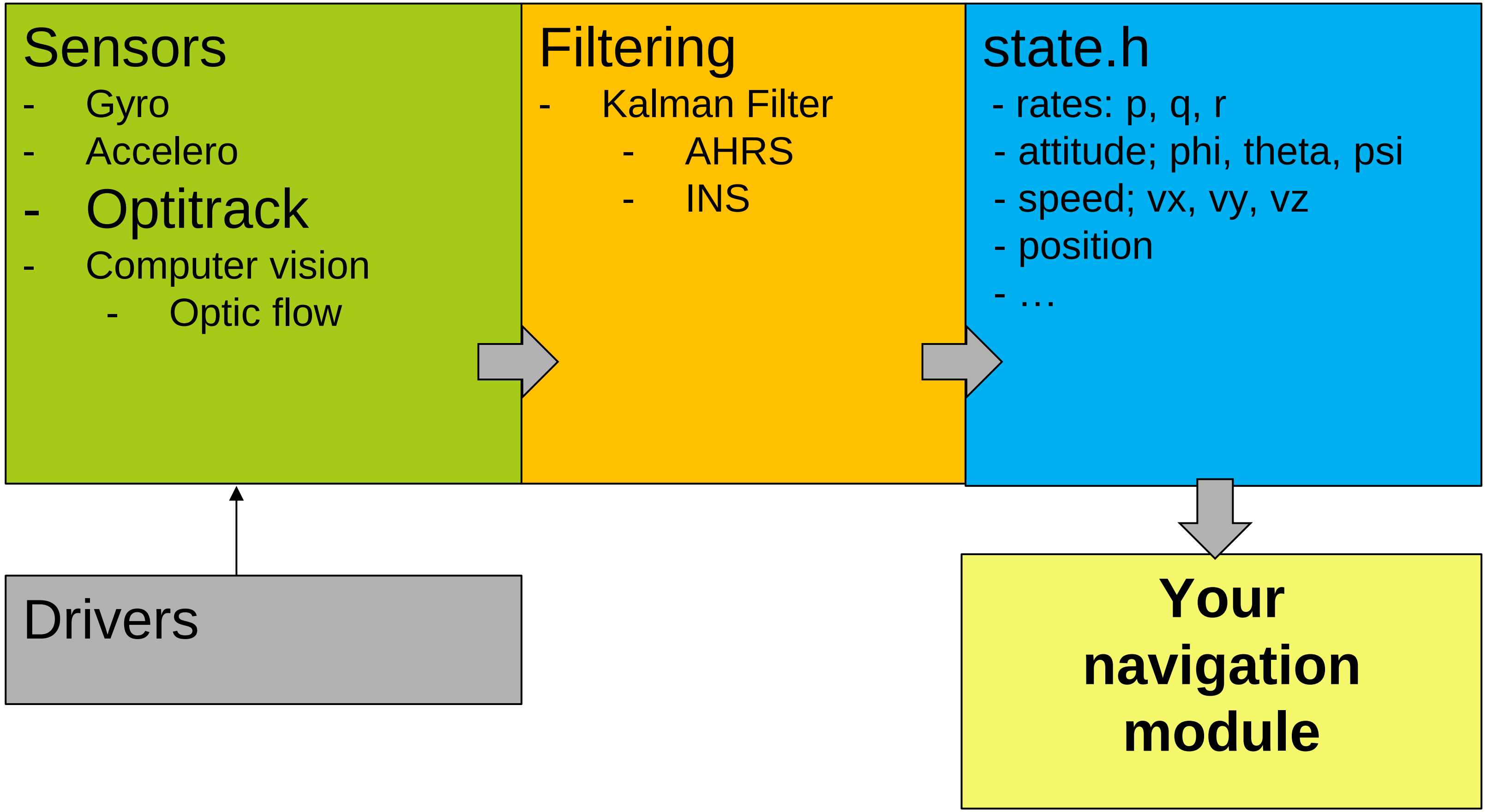


1) Autopilot code: “AP”



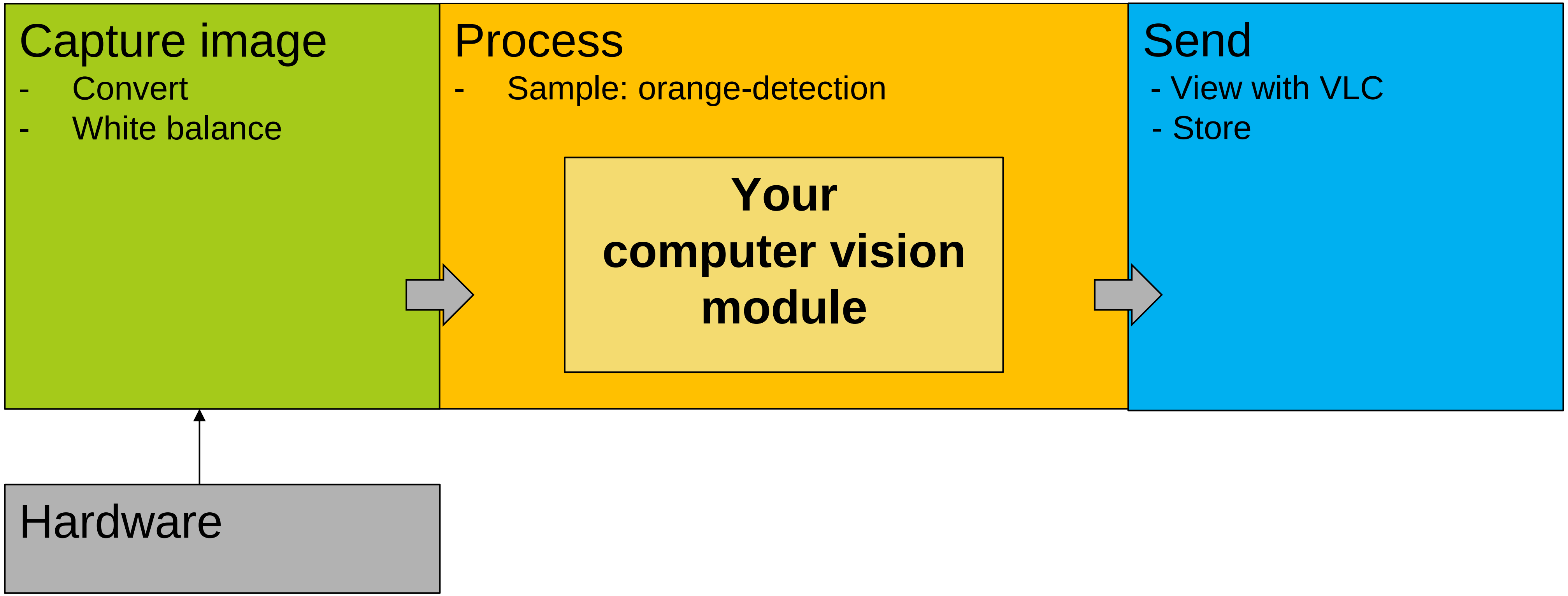
TUDelft

State



AE4317 “getting-started” example code: **orange_avoider.c**

Vision thread

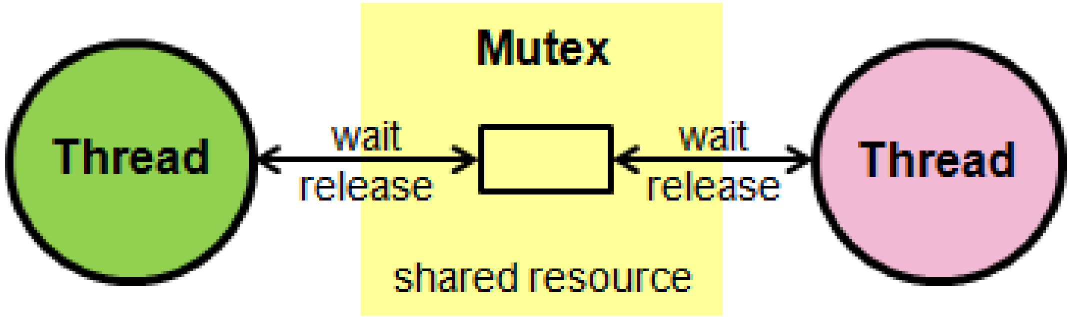


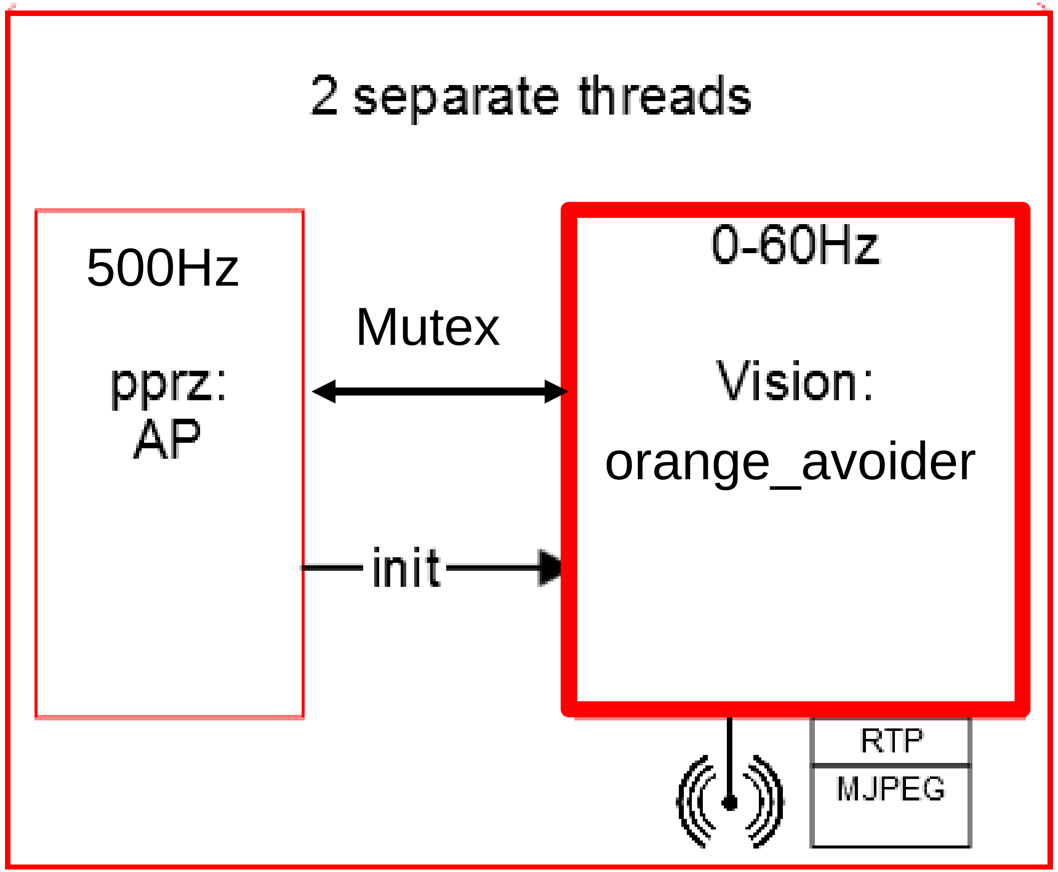
AE4317 Working Example Code: **cv_detect_color_object.c**

Multi-Threading

When 2 threads exchange data, they may not access the memory at the same time!

- Use a Mutex (see orange_avoider)





Get started: orange-avoider

AE4317 “getting-started” example navigation code: **orange_avoider.c**

Autopilot: Light code only! 500Hz!

orange pixels

AE4317 “getting-started” example vision code: **cv_detect_color_object.c**

Vision thread: heavy vision code.

Code

Branch

- Remote: **https://github.com/tudelft/paparazzi.git**
- Branch: git checkout **mavlabCourse2025**
- Select: tudelft/**course_conf.xml**
- Aircraft: **bebop_orange_avoid.xml**
- Target: **NPS** = simulation, **AP** = bebop

- VLC open: **/sw/tools/rtp_viewer/rtp_5000.sdp**

"The Drone"

Get started – organize team

- Follow step-by-step crash-course PDF
- Prototyping algorithms -> Python
  - Use sample images (pdf/intro)
- Run simulation
  - Find xml (airframe / module / ...)
  - Find **cv_detect_color_object.c**
  - Find **orange_avoider.c**

TUDelft

Paparazzi-checklist

you should understand:

- Why are there thread?
- Wat is a MUTEX?
- Where is the PDF to get started?
- Which files should you replace with something better?

Next time:

- **Crash course in C**
- **Collaborative programming**

Schedule

	W1	W2	W3	W4	W5	W6	W7
Form groups							
Install paparazzi							
Try out provided orange module							
All crash course modules							
Devise strategy							
Prototype vision algorithms (scripts / sim)							
Implement control algos in Paparazzi (sim/real)							
Implement vision algos in Paparazzi (sim/real)							
Test, iterate							
FINALIZE CODE FOR COMPETITION							D-DAY

Demo